

Lando SSL Version 2 Well-formedness

November 15, 2023

1 Notation

Lists of t are written $[t]$; $[]_t$ is an empty list of t 's, $x :: xs$ is the list with head x and tail xs , and $[x]$ is the singleton list containing x . $last(l)$ returns the last element of (necessarily non-empty) list l . An optional t is written $t?$; a present option value x is written $[x]$ and a missing option value of type t is written $\cdot_t$. We write $x.y$ for the projection of field y from a node x . In the pattern $x@Foo\{\dots\}$, x is bound to the entire node labeled `Foo`. The pattern $(Foo|Bar)\{\dots\}$ matches nodes labeled either `Foo` or `Bar`. Disjoint union of maps is written $\uplus$, and is undefined if the domains of the maps are not disjoint. If m_1 and m_2 are maps then $m_1 \triangleleft m_2$ (pronounced “ m_1 shadows m_2 ”) is $m_1 \cup \{(k, m_2(k)) | k \in Dom(m_2) - Dom(m_1)\}$. An empty map is written $\emptyset$. A judgement applies only if all premises are fully defined.

2 Raw AST Grammar

This is intended to correspond precisely to the `RawAST` structure built by the current parser, omitting source position tags and comments. The AST corresponds directly to the concrete syntax, with one exception: implicit bodies of `systems` and `subsystems` are converted into explicit ones.

$s \in source$	$\rightarrow$	<code>Source{elems : [elem]}</code>
$e \in elem$	$\rightarrow$	<code>System{name : name, abbrev : name?, explanation : text, indexings : [indexing], body : [elem]}</code>
	$\rightarrow$	<code>Subsystem{name : name, abbrev : name?, inherits : [qname], clientOf : [qname], explanation : text, indexings : [indexing], body : [elem]}</code>
	$\rightarrow$	<code>SubsystemImport{name : qname, abbrev : name?, clientOf : [qname]}</code>
	$\rightarrow$	<code>Component{name : name, abbrev : name?, inherits : [qname], clientOf : [qname], explanation : text, parts : [part]}</code>
	$\rightarrow$	<code>ComponentImport{name : qname, abbrev : name?, clientOf : [qname]}</code>
	$\rightarrow$	<code>Events{name : name, events : [item]}</code>
	$\rightarrow$	<code>Scenarios{name : name, scenarios : [item]}</code>
	$\rightarrow$	<code>Requirements{name : name, requirements : [item]}</code>
	$\rightarrow$	<code>Relation{name : qname, inherits : [qname], clientOf : [qname]}</code>
$x \in indexing$	$\rightarrow$	<code>Indexing{key : text, values : [text]}</code>
$p \in part$	$\rightarrow$	<code>Constraint{text : text}</code>
	$\rightarrow$	<code>Query{text : text}</code>
	$\rightarrow$	<code>Command{text : text}</code>
$i \in item$	$\rightarrow$	<code>Item{id : name, text : text}</code>
$u \in uid$		(base type)
$t \in text$		(base type)
$n \in name$		(base type)
$q \in qname$	$=$	<code>[name]</code>
$c \in comment$		(base type)

3 Well-formedness Judgements

Although the AST does not explicitly include element labels, we assume that each node (in particular each *elem*) has a unique identity with a well-defined notion of equality.

$\Gamma : \text{name} \rightarrow \text{elem}$ is an environment (finite map) from names to their corresponding elements.

$\Phi : \text{elem} \rightarrow \Gamma$ is a finite map from (sub)systems to the environments they generate.

The function $qlook_\Gamma(q)$ resolved a qualified name q starting in environment Γ ; it is defined thus:

$$\begin{aligned} qlook_\Gamma([n]) &= \Gamma(n) \\ qlook_\Gamma(n :: ns) &= qlook_{\Phi(\Gamma(n))}(ns) \end{aligned}$$

$I : \text{elem} \times \text{elem}$ is an inheritance relation between elements, where $(e_1, e_2) \in I$ means e_1 inherits from e_2 . The predicate $nocycles(I)$ holds when I has no cycles.

We assume the existence of a function $qnames(t)$ that returns the list of *qnames* mentioned in *text* t .

A specification s is well-formed if it is possible to find a top-level environment Γ_0 , a global Φ_0 , and a global inheritance relation I_0 (all implicitly threaded everywhere) such that $\vdash s$.

Below we provide a short description of each judgement:

• Judgement 1

- $\Gamma_0 \vdash es \Rightarrow \Gamma_0$: This premise checks the well-formedness of a list of elements es in the context of a top-level environment Γ_0 . It asserts that processing es in Γ_0 results in Γ_0 , which implies that the elements es are self-contained and do not modify the top-level environment.
- $nocycles(I)$: This states that the global inheritance relation I must not contain any cycles.
- The third premise ensures uniqueness of **System** elements within es . It states that if e_1 and e_2 are both **System** elements within es , they must be the same entity ($e_1 = e_2$).
- $\forall e \in es, \text{valid-toplevel}(e)$: This checks that every element e in es satisfies the **valid-toplevel** judgment, meaning each element is valid as a top-level construct.
- $\vdash \text{Source}\{\text{elems} = es\}$: If all the above premises are satisfied, the conclusion states that a **Source** element containing a list of elements es is well-formed. This **Source** element acts as a container or root for the structure defined by the elements in es .

$$\frac{\begin{array}{c} \Gamma_0 \vdash es \Rightarrow \Gamma_0 \quad nocycles(I) \\ \forall e_1, e_2 \in es, e_1 = \text{System}\{\dots\} \wedge e_2 = \text{System}\{\dots\} \implies e_1 = e_2 \quad \forall e \in es, \text{valid-toplevel}(e) \end{array}}{\vdash \text{Source}\{\text{elems} = es\}} \quad (1)$$

• Judgement 2

- $\Gamma \vdash e \Rightarrow \Gamma'$: This premise asserts that a single element e is well-formed in the context of an environment Γ , resulting in a possibly modified environment Γ' .
- $\Gamma \vdash es \Rightarrow \Gamma''$: Similarly, this checks the well-formedness of a list of elements es in the same initial environment Γ , resulting in another possibly modified environment Γ'' . It's assessing the collective impact of the remaining elements in the list after e .
- $\Gamma \vdash e :: es \Rightarrow \Gamma' \uplus \Gamma''$: The conclusion states that if both premises are true, then the entire list formed by concatenating e at the head of es ($e :: es$) is well-formed in the initial environment Γ . The resulting environment after processing this concatenated list is the disjoint union of Γ' and Γ'' (denoted as $\Gamma' \uplus \Gamma''$).

$$\frac{\Gamma \vdash e \Rightarrow \Gamma' \quad \Gamma \vdash es \Rightarrow \Gamma''}{\Gamma \vdash e :: es \Rightarrow \Gamma' \uplus \Gamma''} \quad (2)$$

• **Judgement 3**

- An empty list of elements is always well formed.

$$\frac{}{\Gamma \vdash []_{elem} \Rightarrow \{\}} \quad (3)$$

• **Judgement 4** System well-formed

- $n \neq n_a$: This states that the name n of the **System** element must not be equal to its abbreviation n_a . This ensures distinct identifiers for the full name and the abbreviation.
- $\Gamma \vdash t$: This checks the well-formedness of the **explanation** text t in the current environment Γ .
- $\forall e \in es_b, \text{valid-contains}(e_s, e)$: For every element e in the body es_b of the **System**, this condition asserts that e can be contained within the the **System**
- $\Phi_0(e_s) = \Gamma'$: This premise specifies that the environment generated by the **System** element e_s is Γ'
- $\Gamma' \triangleleft \Gamma \vdash es_b \Rightarrow \Gamma'$: This checks the well-formedness of the body es_b of the **System** in an environment that is a combination of Γ' (specific to the **System**) and Γ (the broader environment), with Γ' taking precedence in case of overlaps.
- If all the above premises are satisfied, then the **System** element e_s , defined with a name n , an optional abbreviation n_a , an explanation t , and a body es_b (along with other unspecified properties represented by $\dots$), is well-formed in the environment Γ . Furthermore, the resulting environment from processing this **System** element maps both n and n_a to e_s .

$$\frac{n \neq n_a \quad \Gamma \vdash t \quad \forall e \in es_b, \text{valid-contains}(e_s, e) \quad \Phi_0(e_s) = \Gamma' \quad \Gamma' \triangleleft \Gamma \vdash es_b \Rightarrow \Gamma'}{\Gamma \vdash e_s @ \text{System}\{\text{name} = n, \text{abbrev} = [n_a], \text{explanation} = t, \text{body} = es_b, \dots\} \Rightarrow \{n \mapsto e_s, n_a \mapsto e_s\}} \quad (4)$$

• **Judgement 5** System well-formed

- The same as Judgement 4 except for systems without the abbreviation n_a

$$\frac{\Gamma \vdash t \quad \forall e \in es_b, \text{valid-contains}(e_s, e) \quad \Phi_0(e_s) = \Gamma' \quad \Gamma' \triangleleft \Gamma \vdash es_b \Rightarrow \Gamma'}{\Gamma \vdash e_s @ \text{System}\{\text{name} = n, \text{abbrev} = \cdot_{name}, \text{explanation} = t, \text{body} = es_b, \dots\} \Rightarrow \{n \mapsto e_s\}} \quad (5)$$

• **Judgement 6** Subsystem well-formed

- This judgment defines the well-formedness criteria for a **Subsystem** element in the Lando language.
- The name and the abbreviated name must be different.
- e_s must have only valid inherit relations qs_i
- e_s must contain only valid elements es_b
- e_s must have only valid client relations qs_c
- $\Phi_0(e_s) = \Gamma'$: This premise specifies that the environment generated by the **Subsystem** element e_s is Γ'
- $\Gamma' \triangleleft \Gamma \vdash es_b \Rightarrow \Gamma'$: The well-formedness of the body es_b of the **Subsystem** is checked in an environment that combines Γ' and Γ , with Γ' taking precedence.
- If all those conditions are met, then the subsystem is well formed.

$$\frac{n \neq n_a \quad \forall q \in qs_i, \text{valid-inherit}(e_s, \text{qlook}_\Gamma(q)) \quad \forall q \in qs_c, \text{valid-client}(e_s, \text{qlook}_\Gamma(q))}{\Gamma \vdash t \quad \forall e \in es_b, \text{valid-contains}(e_s, e) \quad \Phi_0(e_s) = \Gamma' \quad \Gamma' \triangleleft \Gamma \vdash es_b \Rightarrow \Gamma'} \quad (6)$$

$$\Gamma \vdash e_s @ \text{Subsystem} \left\{ \begin{array}{l} \text{name} = n, \text{abbrev} = [n_a], \text{inherits} = qs_i, \\ \text{clientOf} = qs_c, \text{explanation} = t, \text{body} = es_b, \dots \end{array} \right\} \Rightarrow \{n \mapsto e_s, n_a \mapsto e_s\}$$

• **Judgement 7** Subsystem well-formed

- The same as Judgement 6 except for subsystems without the abbreviation n_a

$$\frac{\forall q \in qs_i, \text{valid-inherit}(e_s, \text{qlook}_\Gamma(q)) \quad \forall q \in qs_c, \text{valid-client}(e_s, \text{qlook}_\Gamma(q))}{\Gamma \vdash t \quad \forall e \in es_b, \text{valid-contains}(e_s, e) \quad \Phi_0(e_s) = \Gamma' \quad \Gamma' \triangleleft \Gamma \vdash es_b \Rightarrow \Gamma'} \quad (7)$$

$$\Gamma \vdash e_s @ \text{Subsystem} \left\{ \begin{array}{l} \text{name} = n, \text{abbrev} = \cdot_{name}, \text{inherits} = qs_i, \\ \text{clientOf} = qs_c, \text{explanation} = t, \text{body} = es_b, \dots \end{array} \right\} \Rightarrow \{n \mapsto e_s\}$$

• **Judgement 8** Subsystem import

- $\text{qlook}_{\Gamma_0}(q) = e$: This premise asserts that the qualified name q resolves to an element e in the top-level environment Γ_0 .
- $\forall q \in qs_c, \text{valid-client}(e, \text{qlook}_\Gamma(q))$: For each qualified name q in the list qs_c (which represents elements for which the imported subsystem is a client), the element corresponding to q must have a valid client relationship with e (the element resolved from q).
- If the above premises are satisfied, then the **SubsystemImport** element e_s , defined with a reference name q , an optional abbreviation n_a , and a list of clients qs_c , is well-formed in the environment Γ . The resulting environment from processing this import maps the abbreviation n_a to the **SubsystemImport** element e_s .

$$\frac{\text{qlook}_{\Gamma_0}(q) = e \quad \forall q \in qs_c, \text{valid-client}(e, \text{qlook}_\Gamma(q))}{\Gamma \vdash e_s @ \text{SubsystemImport}\{\text{name} = q, \text{abbrev} = [n_a], \text{clientOf} = qs_c\} \Rightarrow \{n_a \mapsto e_s\}} \quad (8)$$

• **Judgement 9** Subsystem import (no n_a)

- Same as Judgement 8, except for imported subsystems without an abbreviation n_a . As a result, we use the last part of the qualified name as the abbreviation of the subsystem.

$$\frac{\text{qlook}_{\Gamma_0}(q) = e \quad \forall q \in qs_c, \text{valid-client}(e, \text{qlook}_\Gamma(q))}{\Gamma \vdash e_s @ \text{SubsystemImport}\{\text{name} = q, \text{abbrev} = \cdot_{name}, \text{clientOf} = qs_c\} \Rightarrow \{last(q) \mapsto e_s\}} \quad (9)$$

• **Judgement 10** Component well-formed

- Distict name and the abbreviated name
- Valid client and valid inherit relationships
- The explanation t must be well formed
- Each part p must be well formed
- If all those premises are met, the component e_c is well formed

$$\frac{n \neq n_a \quad \forall q \in qs_i, \text{valid-inherit}(e_c, \text{qlook}_\Gamma(q)) \quad \forall q \in qs_c, \text{valid-client}(e_c, \text{qlook}_\Gamma(q)) \quad \Gamma \vdash t \quad \forall p \in ps, \Gamma \vdash p}{\Gamma \vdash e_c @ \text{Component} \left\{ \begin{array}{l} \text{name} = n, \text{abbrev} = [n_a], \text{inherits} = qs_i, \\ \text{clientOf} = qs_c, \text{explanation} = t, \text{parts} = ps \end{array} \right\} \Rightarrow \{n \mapsto e_c, n_a \mapsto e_c\}} \quad (10)$$

• **Judgement 11** Component well formed (no n_a)

- Same as Judgement 10, except for components without an abbreviated name.
- Unclear what *a missing option for name* means (`abbrev = .name`)

$$\frac{\begin{array}{c} \forall q \in qs_i, \text{valid-inherit}(e_c, \text{qlook}_\Gamma(q)) \\ \forall q \in qs_c, \text{valid-client}(e_c, \text{qlook}_\Gamma(q)) \quad \Gamma \vdash t \quad \forall p \in ps, \Gamma \vdash p \end{array}}{\Gamma \vdash e_c @ \text{Component} \left\{ \begin{array}{l} \text{name} = n, \text{abbrev} = \text{.name}, \text{inherits} = qs_i, \\ \text{clientOf} = qs_c, \text{explanation} = t, \text{parts} = ps \end{array} \right\} \Rightarrow \{n \mapsto e_c\}} \quad (11)$$

• **Judgement 12** Constraint well-formed

- If the text t is well-formed, a **Constraint** is also well-formed

$$\frac{\Gamma \vdash t}{\Gamma \vdash \text{Constraint}\{\text{text} = t\}} \quad (12)$$

• **Judgement 13** Query well-formed

- If the text t is well-formed, a **Query** is also well-formed

$$\frac{\Gamma \vdash t}{\Gamma \vdash \text{Query}\{\text{text} = t\}} \quad (13)$$

• **Judgement 14** Command well-formed

- If the text t is well-formed, a **Command** is also well-formed

$$\frac{\Gamma \vdash t}{\Gamma \vdash \text{Command}\{\text{text} = t\}} \quad (14)$$

• **Judgement 15** Component import

- Qualified name q resolves to an element e in the top level environment
- $\forall q \in qs_c, \text{valid-client}(e, \text{qlook}_\Gamma(q))$: For each qualified name q in the list qs_c (representing elements for which the imported component is a client), the element corresponding to q must have a valid client relationship with e .
- If the above defined premises are satisfied, the imported component is well-formed.

$$\frac{\text{qlook}_{\Gamma_0}(q) = e \quad \forall q \in qs_c, \text{valid-client}(e, \text{qlook}_\Gamma(q))}{\Gamma \vdash e_c @ \text{ComponentImport}\{\text{name} = q, \text{abbrev} = \lceil n_a \rceil, \text{clientOf} = qs_c\} \Rightarrow \{n_a \mapsto e_c\}} \quad (15)$$

• **Judgement 16** Component import without n_a

- Like Judgement 15, except covering cases where the imported component does not have an abbreviated name.
- **TODO: what do we do with the name?**

$$\frac{\text{qlook}_{\Gamma_0}(q) = e \quad \forall q \in qs_c, \text{valid-client}(e, \text{qlook}_\Gamma(q))}{\Gamma \vdash e_c @ \text{ComponentImport}\{\text{name} = q, \text{abbrev} = \text{.name}, \text{clientOf} = qs_c\} \Rightarrow \{\text{last}(q) \mapsto e_c\}} \quad (16)$$

- **Judgement 17** Events well-formed

- If a list of events is is well formed, then an element e of type *Events* is well formed

$$\frac{\Gamma \vdash is \Rightarrow \Gamma'}{\Gamma \vdash e@Events\{name = n, events = is\} \Rightarrow \{n \mapsto e\} \uplus \Gamma'} \quad (17)$$

- **Judgement 18** Scenarios well-formed

- If a list of scenarios is is well formed, then an element e of type *Scenarios* is well formed

$$\frac{\Gamma \vdash is \Rightarrow \Gamma'}{\Gamma \vdash e@Scenarios\{name = n, scenarios = is\} \Rightarrow \{n \mapsto e\} \uplus \Gamma'} \quad (18)$$

- **Judgement 19** Requirements well-formed

- If a list of scenarios is is well formed, then an element e of type *Requirements* is well formed

$$\frac{\Gamma \vdash is \Rightarrow \Gamma'}{\Gamma \vdash e@Requirements\{name = n, requirements = is\} \Rightarrow \{n \mapsto e\} \uplus \Gamma'} \quad (19)$$

- **Judgement 20**

- An empty list of items is always well-formed

$$\overline{\Gamma \vdash []_{item} \Rightarrow \{\}} \quad (20)$$

- **Judgement 21**

- See Judgement 2 for detailed meaning of the premises.
- $\Gamma \vdash i :: is \Rightarrow \Gamma'' \uplus \Gamma'''$: The conclusion states that if both premises are true, then the entire list formed by concatenating i and is is well-formed in the initial environment Γ . The resulting environment after processing this concatenated list is the disjoint union of Γ'' and Γ''' (denoted as $\Gamma'' \uplus \Gamma'''$).

$$\frac{\Gamma \vdash i \Rightarrow \Gamma'' \quad \Gamma \vdash is \Rightarrow \Gamma'''}{\Gamma \vdash i :: is \Rightarrow \Gamma'' \uplus \Gamma'''} \quad (21)$$

- **Judgement 22** Item well-formed

- If the text t is well-formed, then an item i is well-formed.

$$\frac{\Gamma \vdash t}{\Gamma \vdash i@Item\{id = n, text = t\} \Rightarrow \{n \mapsto i\}} \quad (22)$$

- **Judgement 23** Relation well-formed

- Qualified name q must be resolved to an element e
- All inherit relations are valid

- All client relations are valid
- A well-formed relation does not add a new element to Γ , it only needs to be valid within the environment

$$\frac{qlook_{\Gamma}(q) = e \quad \forall q \in qs_i, \text{valid-inherit}(e, qlook_{\Gamma}(q)) \quad \forall q \in qs_c, \text{valid-client}(e, qlook_{\Gamma}(q))}{\Gamma \vdash \text{Relation}\{\text{name} = q, \text{inherits} = qs_i, \text{clientOf} = qs_c\} \Rightarrow \{\}} \quad (23)$$

• **Judgement 24** Text well-formed

- This judgment defines the well-formedness criteria for a piece of text t focusing on the resolution of names within that text. It's a rule that ensures every name mentioned in the text t corresponds to an existing element in the current environment.
- $\forall n_t \in qnames(t), \exists e, \Gamma(n_t) = e$: For each name n_t that appears within the text t (as identified by the function $qnames(t)$), there must exist an element e such that n_t resolves to e in the environment Γ . This means that every name referenced in t must have a corresponding, defined element in the current environment.
- $\Gamma \vdash t$: If the above premise is satisfied, then the text t is considered well-formed in the environment Γ . This conclusion states that the text is valid within the context of the environment, assuming that all names it mentions are properly accounted for and linked to existing elements.

$$\frac{\forall n_t \in qnames(t), \exists e, \Gamma(n_t) = e}{\Gamma \vdash t} \quad (24)$$

• **Judgement 25**

- $\text{valid-toplevel}((\text{System}|\text{Subsystem}|\dots)\{\dots\})$: The rule states that the described elements are always valid as top-level elements. The $\{\dots\}$ indicates that the specific contents of these elements are not relevant to this judgment

$$\text{valid-toplevel}((\text{System}|\text{Subsystem}|\text{Component}|\text{Events}|\text{Scenarios}|\text{Requirements}|\text{Relation})\{\dots\}) \quad (25)$$

• **Judgement 26**

- This rule states that a **System** can contain only **Subsystems**, imported **Subsystems** or **Relations**

$$\text{valid-contains}(\text{System}\{\dots\}, (\text{Subsystem}|\text{SubsystemImport}|\text{Relation})\{\dots\}) \quad (26)$$

• **Judgement 27**

- This rule states that a **SubSystem** can contain only **Subsystems**, imported **Subsystems**, **Components** or imported **Components**

$$\text{valid-contains}(\text{Subsystem}\{\dots\}, (\text{Subsystem}|\text{SubsystemImport}|\text{Component}|\text{ComponentImport})\{\dots\}) \quad (27)$$

- **Judgement 28**

- This rule states that a **SubSystem** can contain only **Scenarios**, **Requirements**, **Events** or **Relations**

$$\frac{}{\text{valid-contains}(\text{Subsystem}\{\dots\}, (\text{Scenarios} | \text{Requirements} | \text{Events} | \text{Relation})\{\dots\})} \quad (28)$$

- **Judgement 29**

- $(e_1, e_2) \in I_0$: This premise states that there is an inheritance relationship between two elements e_1 and e_2 in the global inheritance relation I_0 . In other words, e_1 inherits from e_2 .
- $\text{valid-inherit}(e_1 @ \text{Component}\{\dots\}, e_2 @ \text{Component}\{\dots\})$: If the above premise is satisfied, this conclusion asserts that the inheritance relationship between e_1 and e_2 is valid, where both e_1 and e_2 are **Component** elements. The $\{\dots\}$ again implies that the specific contents or attributes of the **Component** elements are not relevant to this judgment.

$$\frac{(e_1, e_2) \in I_0}{\text{valid-inherit}(e_1 @ \text{Component}\{\dots\}, e_2 @ \text{Component}\{\dots\})} \quad (29)$$

- **Judgement 30**

- This rule describes which elements can be a client of an (imported) subsystem.

$$\frac{}{\text{valid-client}((\text{Subsystem} | \text{SubsystemImport})\{\dots\}, (\text{Subsystem} | \text{SubsystemImport} | \text{Component} | \text{ComponentImport})\{\dots\})} \quad (30)$$

- **Judgement 31**

- This rule describes which elements can be a client of an (imported) component.

$$\frac{}{\text{valid-client}((\text{Component} | \text{ComponentImport})\{\dots\}, (\text{Subsystem} | \text{SubsystemImport} | \text{Component} | \text{ComponentImport})\{\dots\})} \quad (31)$$